# Data Analysis Programming
# Semester 2026-II
# Course Project Description
# Natural Hazard Monitoring Systems

**Eng. Carlos Andrés Sierra, M.Sc.**
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

## Overview

The course project of *Data Analysis Programming* is a semester-long, team-based data engineering and analytics project. Every team builds a system with the same general objective:

> *Design and implement a multi-source data ingestion and analysis system capable of integrating historical and continuously updated information about a natural hazard, detecting new events or changes, and producing analytical indicators about the current situation.*

All teams share the same technical contract, the same conceptual data model, and the same evaluation rubric, but each team works on a **different natural hazard** (earthquakes, floods, wildfires, tropical cyclones, extreme weather, volcanoes, landslides, tsunamis, or compound weather hazards). Sharing the architecture while varying the domain makes the nine projects academically equivalent and comparable, and lets the course teach data modeling and abstraction instead of nine unrelated databases.

The project is organized in four workshops and a final delivery:

i. **Workshop #1: Hazard Definition, Source Validation, and Data Contract** — Source research, real API/file samples, canonical schema mapping, and repository setup.

---

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

ii. **Workshop #2: Architecture and Historical Ingestion** — Docker stack, database schema, and `python ingest.py -mode historical` with normalization, validation, missing-value handling, and quarantine.

iii. **Workshop #3: Incremental Ingestion, Deduplication, and Event Identification** — Watermarks, idempotent polling, retries, scheduling, duplicate handling, and cross-source event matching.

iv. **Workshop #4: Analytics and Monitoring Dashboard** — Temporal and geographic analysis, historical-versus-current comparison, the hazard indicator, and the Plotly Dash dashboard.

v. **Final Project: Integration, Report, and Defense** — Fully reproducible product, final report, live demonstration, and individual defense.

## Common Technical Specification

**Historical ingestion**

- At least **two historical sources** with **different formats or APIs** (e.g., a REST API returning GeoJSON and a bulk CSV archive).
- **Several years of data** (at least five years where available), loaded through an initial bulk ingestion.

**Incremental ingestion**

- At least **two continuously or periodically updated sources**.
- Scheduled polling, detection of new (and changed) records, duplicate handling, and incremental storage.
- In this course, *incremental ingestion* means polling an operationally updated source and ingesting only the observations or events that appeared or changed since the previous execution. Several hazard sources are not literal streaming APIs; implementing the incremental logic is precisely the learning objective.

**Data processing (required by every team)**

- Schema normalization into the canonical model
- Data validation (ranges, types, coordinates, timestamps)
- Missing-value handling with a documented strategy per field
- Duplicate detection (within a source and across ingestion runs)
- Event identification (grouping records that describe the same real-world event)
- Source integration (cross-source matching)

- Temporal analysis
- Geographic analysis
- Historical-versus-current comparison
- A monitoring/analytical dashboard

### Command-line contract

Every project must expose the same entry point:

```
python ingest.py --mode historical  [--start 2010-01-01 --end 2025-12-31]
python ingest.py --mode incremental
python ingest.py --mode status        # watermarks, last runs, row counts
```

- `historical` populates the initial database and may be re-run safely (idempotent).
- `incremental` retrieves only the interval `last_ingestion` → `NOW`, never downloading everything again.
- Running either mode twice in a row must not create duplicate rows.

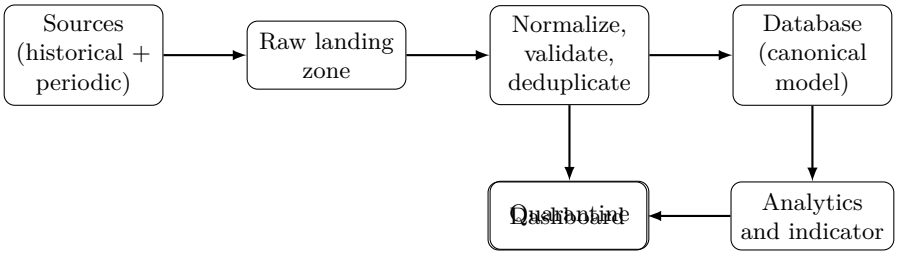
### Common conceptual data model

Every team implements an abstraction analogous to the following; only the domain tables change.

| Table | Key fields | Purpose |
|---|---|---|
| `source` | `source_id`, name, url, format, license | Catalog of every data source |
| `ingestion_run` | `run_id`, mode, started, finished, status, rows_in, rows_new, rows_rejected | Auditable record of each execution |
| `ingestion_state` | `source_id`, watermark, last_success | Incremental state (high-water mark) per source |
| `event` | `event_id`, source, event_type, timestamp, latitude, longitude, severity, raw_reference | Canonical observation/event, one row per source record |
| `event_group` | `group_id`, `event_id` | Links source records that describe the same real-world event |
| `rejected_record` | `run_id`, source, reason, payload | Quarantine for records failing validation |
| Domain tables | e.g., `earthquake`, `fire_detection` | Hazard-specific attributes referencing `event` |

Required attributes of `event`: `event_id`, `source`, `event_type`, `timestamp` (UTC), `latitude`, `longitude`, `severity` (with its unit), and `raw_reference` (the original identifier or URL, plus a content hash used for change detection).

**Reference architecture**




The infrastructure runs locally with Docker Compose: a relational database (PostgreSQL, with PostGIS recommended; SQLite/GeoPackage is acceptable only if justified), an orchestrator for the incremental schedule (Apache Airflow DAG or an equivalent documented scheduler), the ingestion application, and the Plotly Dash dashboard. A raw landing zone (files kept exactly as received, named `source_YYYYMMDD_HHMMSS.ext`) preserves provenance and allows re-processing.

## The Nine Natural Hazard Projects

Each project below shares the contract above; teams are assigned a hazard and may not define their own hazard. Detailed source suggestions, verified access notes, and analytical goals per hazard are given in the *Team Project Assignment* document. Some required sources (NOAA Storm Events, NWS) cover only the United States; the geographic scope of each team must be consistent with the coverage of its sources.

| # | Hazard system | Main analytical output | Main programming challenge |
|---|---|---|---|
| 1 | Earthquake Activity | Frequency, magnitude/depth distributions, unusual activity | Deciding whether records from different catalogs are the same earthquake |
| 2 | Flood Monitoring and Risk | Flood Risk Index (LOW / MEDIUM / HIGH) | Combining temporal rainfall with geographic flood events |
| 3 | Wildfire Monitoring | Anomalous fire activity per region | Geospatial and temporal aggregation of satellite detections |
| 4 | Tropical Cyclone Monitoring | Comparison of current storm against similar historical storms | Track processing and trajectory comparison |
| 5 | Extreme Weather | NORMAL / UNUSUAL / EXTREME per measurement | Anomaly detection with historical distributions |
| 6 | Volcanic Activity | Volcano Activity Indicator | Joining events with static geographic entities |

| # | Hazard system | Main analytical output | Main programming challenge |
|---|---|---|---|
| 7 | Landslide Monitoring and Risk | Landslide Risk Indicator per region | Spatial joins between measurements and historical events |
| 8 | Tsunami Monitoring | Potential tsunami relevance of an earthquake | Correlating earthquakes with tsunami information in space and time |
| 9 | Multi-Hazard Weather Impact | Compound Hazard Index per region | Temporal and spatial correlation of several event types |

## Project Scope and Objectives

- **Multi-Source Integration:** Reconcile heterogeneous sources (different formats, units, time zones, identifiers, and coordinate conventions) into one canonical model.
- **Historical and Incremental Ingestion:** Implement bulk loading and state-based incremental loading with watermarks, idempotency, pagination, rate-limit handling, and retries.
- **Data Quality:** Validate, quarantine, and report on problems found in real data, and expose data-quality metrics in the system.
- **Event Identification:** Determine when different records refer to the same real-world event, and justify the matching rule (time window, distance threshold, attribute similarity).
- **Analytics:** Produce temporal and geographic indicators and compare current conditions with the historical baseline.
- **Monitoring Dashboard:** Communicate the current situation, new events, and the hazard indicator to a defined functional user.
- **Responsible Communication:** Present indicators as monitoring and correlation aids. Teams must *not* claim to predict hazards (for example, tsunamis), and must state the limitations of each indicator.
- **Engineering Practice:** Configuration outside code, structured logging, automated tests, and a reproducible repository.

**Final Project Implementation Steps**

1. **Full Pipeline Integration and Validation:**
   - Consolidate all workshop deliverables into a single coherent system.
   - From a clean state, run `-mode historical`, then several `-mode incremental` executions, and verify that the dashboard reflects the result.
   - Prove idempotency: repeating any execution produces no duplicates and identical counts.
   - Resolve schema inconsistencies and update the data model documentation.
2. **Robustness and Production Readiness:**
   - Retry with exponential backoff, timeouts, and graceful handling of source outages and schema changes.
   - Recovery after a failed run without losing or duplicating data (the watermark advances only after a successful commit).
   - Structured logging and the `ingestion_run` audit trail.
   - Automated tests (`pytest`) for normalization, validation, deduplication, event matching, and the watermark logic, using recorded fixtures instead of live calls.
3. **Analytics and Dashboard Finalization:**
   - Finalize the hazard indicator with documented, parameterized rules and a sensitivity discussion.
   - Complete the dashboard: recent events map, time series, historical-versus-current panel, indicator, and a data-health panel (last run, watermark age, rejected records).
   - Review usability from the functional user's perspective defined in Workshop #1.
4. **Comprehensive Project Documentation:**
   - Integrated final report covering the whole lifecycle (see below).
   - Document how the data model and pipeline evolved across workshops, and a data-quality narrative of real issues found and how they were solved.
5. **Repository Finalization and Reproducibility:**
   - Clean repository with a complete `README.md`: hazard description, team members, architecture, setup, and exact commands for historical load, incremental runs, and dashboard launch.
   - Tag the final commit `v1.0-final`.
   - Verify reproduction from a fresh clone following only the `README.md`.
6. **Presentation and Defense:**

- Live demonstration: a historical load, an incremental run that detects at least one new record, and the dashboard.
- Technical questions on any component; individual understanding is evaluated, not only the collective product.

### Deliverables and Documentation

- **Functional System:** Dockerized pipeline with working `historical`, `incremental`, and `status` modes.
- **Database and Data Dictionary:** Canonical schema, domain tables, and a data dictionary with units and source mappings.
- **Monitoring Dashboard** (`localhost:8050`) reading exclusively from the database or its analytical views.
- **Final Report (PDF):** Executive summary; hazard and functional user; sources and licenses; architecture; data model; historical and incremental design (watermarks, idempotency, duplicate strategy); data-quality report; event identification and source integration; analytical methods and indicator definition; dashboard documentation; results and limitations; lessons learned; future work.
- **Complete Repository** with source code, Docker configuration, tests, fixtures, small data samples, and workshop deliverables in labeled folders.
- **Presentation Materials** and live demonstration.

### Submission Requirements

- Submit the Final Report as a single PDF through the designated course platform.
- Include the GitHub repository link with the tagged commit `v1.0-final`.
- All documentation must be in **English** and follow professional technical writing standards.
- Cite every source, API, dataset, license, and method used; respect each source's terms of use, attribution requirements, and rate limits. Never commit API keys; use environment variables and a `.env.example` file.
- The system must be demonstrable live; partial or non-functional systems are evaluated accordingly.

**Final Project Evaluation Rubric (100 points)**

The final delivery is graded with the same rubric for all nine teams. Each workshop is graded separately on 20 points with its own rubric (given in the workshop document), and every grade is converted to its weight in the course grade (see Milestones).

| **Criterion** | **Points** |
|---|---|
| Historical ingestion (two sources, different formats, multi-year, idempotent) | 15 |
| Incremental ingestion (watermarks, polling, new/changed detection, retries) | 15 |
| Data modeling and normalization (canonical model, domain tables, data dictionary) | 10 |
| Validation, missing values, and duplicate handling (including quarantine) | 10 |
| Event identification and source integration (matching rule and evaluation) | 10 |
| Temporal and geographic analysis; historical-versus-current comparison | 15 |
| Hazard indicator (definition, parameterization, limitations) | 5 |
| Dashboard and communication to the functional user | 10 |
| Engineering quality (tests, logging, configuration, reproducibility, documentation) | 10 |
| **Total** | **100** |

Individual understanding adjusts each member's grade: a member who cannot explain and defend the components they claim may receive a lower individual score than the team score.

**Milestones**

| **Milestone** | **Focus** | **Weight in course grade** |
|---|---|---|
| Workshop #1 | Sources and data contract | 10% |
| Workshop #2 | Historical ingestion | 10% |
| Workshop #3 | Incremental ingestion and integration | 10% |
| Workshop #4 | Analytics and dashboard | 10% |
| Final Project | Integration, report, defense | 30% |

*The nine hazards are different, but the engineering problem is the same: bring messy, heterogeneous, ever-changing data into one trustworthy model and turn it into a clear picture of what is happening now compared with what has happened before. Good luck!*